Senthen Velmurugan

Dallas, TX
[redacted] senthenvelmurugan@gmail.com sent-hen.github.io

Education

Georgia Institute of Technology **In Progress**
Master of Science in Computer Science (OMSCS) *Atlanta, GA*

The University of Texas at Dallas **Completed**
Bachelor of Science in Computer Science *Dallas, TX*

Experience

KloudGin **May 2024 – August 2024**
Machine Learning Engineering Intern *Remote*

- ◇ Joined the Generative AI team for a three-month internship to explore whether retrieval-augmented generation (RAG) could speed up the resolution of incoming customer cases
- ◇ Proposed and built a multi-source RAG system that retrieves semantically similar historical cases from the ticketing corpus, mines accepted solutions and discussions from the internal forum, and synthesizes suggested next actions for the support engineer
- ◇ Built the ingestion pipeline (document normalization, chunking, metadata tagging) with hybrid retrieval (vector + keyword, with a BM25 fallback) and cross-encoder re-ranking for relevance
- ◇ Framed baselines before experimenting and compared alternatives with data (chunking strategies, embedding models, BM25 fallback, cross-encoder re-rankers); when an early version overfit to long threads, diagnosed it and retuned chunking and metadata filters to improve top-k precision
- ◇ Implemented a light evaluation harness: offline retrieval-quality metrics, spot checks of suggestion helpfulness, and a simple A/B feedback loop with support agents
- ◇ Added guardrails for privacy (tenant-scoped search, PII scrubbing) and safe prompting, and exposed the system through a clean API with concise documentation so other teams could integrate it
- ◇ Took it from proof of concept to rollout, with a 20% increase in first-contact resolution over 12 weeks of use; wrote design notes and worked directly with stakeholders throughout

GBC Computers **May 2023 – August 2023**
Cybersecurity and Systems Intern *Mabank, TX*

- ◇ Optimized network performance using PRTG as the primary monitoring tool, supplemented by Wireshark for traffic analysis and troubleshooting bottlenecks
- ◇ Secured client infrastructure by configuring firewall rules with pfSense, while integrating Snort for intrusion detection and endpoint security measures for additional protection
- ◇ Performed vulnerability assessments using Nessus to identify critical risks, leveraging OpenVAS and manual security audits to validate findings and apply necessary patches

Projects – Client & Web

Campbell Woodworks: Storefront for a Family Woodshop | *Jekyll, Liquid, HTML, CSS, Python* **2026**

- ◇ Built a storefront for a family woodshop in Athens, TX, whose work lived only in Facebook posts behind a login and whose pieces mostly had no name or price, so buyers could find and browse it between farmers-market days
- ◇ Designed around how the shop actually sells (by phone, not checkout): call and email buttons on every piece instead of a cart
- ◇ Made the shop, gallery, and detail pages all generate from one Markdown file per piece, with category filters, covering 23 pieces photographed from their page and market booth
- ◇ Published it as a static Jekyll site on GitHub Pages, so there is nothing for the shop to run or pay for; next, prices and stock as the shop confirms them, and the maker's own story

Personal Website: sent-hen.github.io | *Jekyll, Liquid, HTML, CSS, JavaScript* **2023 – In Progress**

- ◇ Built a personal site with no server, database, or build pipeline to maintain, published as a static Jekyll site from the main branch through GitHub Pages
- ◇ Drove every list from small data files, so adding a project or an item is a one-file change, and cut the site back repeatedly to only what is used
- ◇ Drew the home-page animations (0/1 rain, golden spiral, Sierpinski triangle, ripple) as generated SVG and plain JavaScript with no libraries, with a Hi/Lo switch that shows still line art instead

Projects – Backend & Infrastructure

Metal-to-Pod: Self-Hosted CI/CD from VMware to Kubernetes | *Kubernetes, Go, Jenkins, Prometheus* **2026**

- ◇ Built a commit-to-pod delivery path with no cloud services across 4 VMware VMs (9 GB total, sized for a 16 GB laptop), provisioned from a single env file with Vagrant and shell
- ◇ Bootstrapped a kubeadm cluster on containerd and Calico, with namespaces, Pod Security, default-deny network policy, and quotas, and fixed the dual-NIC node IP and CNI interface issues explicitly
- ◇ Triggered every build from a git post-receive hook into Jenkins configured entirely as code, pushing to a private registry with TLS from a self-run CA
- ◇ Made releases safe to automate with a multi-stage build that gates on unit tests and ships a ∼10 MB distroless, non-root image tagged with the commit SHA, deployed by a namespace-scoped ServiceAccount instead of cluster-admin
- ◇ Caught bad releases automatically with an external smoke test that confirms the new commit is the one serving, and rolls back if the rollout or smoke test fails
- ◇ Ran a Go delivery-promise service (3 distribution centers, 18 lanes, carrier cutoffs and holidays) against 99.5% availability and 99% under 300 ms SLOs, with burn-rate alerts unit-tested in promtool, 8 failure drills, 5 runbooks, and 7 ADRs

Projects – Information Security

Personal VPN | *Terraform, Go, Bash, WireGuard, Linux, nftables* **2026 – In Progress**

- ◇ Protected traffic on untrusted networks (café and hotel Wi-Fi) without handing that trust to a commercial VPN, by owning every layer from server to client
- ◇ Provisioned a hardened WireGuard server on Hetzner for about €4/month with a single `terraform apply`, using cloud-init for SSH hardening, nftables NAT over IPv4 and IPv6, Unbound DNS, fail2ban, and unattended security upgrades
- ◇ Kept private keys from ever crossing the network by having each device generate its own key and share only the public half, and by generating the server key on first boot, so Terraform state holds no secrets
- ◇ Built `vpnctl`, a Go CLI for device key generation and WireGuard client configs, covered by 11 Go unit tests, plus a `vpn-peer` tool to add or remove devices without rebuilding the server
- ◇ Reduced the attack surface to one UDP port by default-denying inbound traffic in nftables and the cloud firewall, and limiting key-only SSH to listed admin CIDRs
- ◇ Documented a threat model stating what the VPN protects against and what it does not (privacy on untrusted networks, not anonymity), plus a manual runbook that mirrors every automated step; next: a Windows service with a WFP kill switch and a Tauri UI

Honeypot & Attack Map | *Terraform, AWS, Bash, Docker, Cowrie, Vector* **2026 – In Progress**

- ◇ Set out to see who attacks an exposed server and what they try, and to turn that data into lessons for defending real systems, without letting the bait become a launch point
- ◇ Deployed a Cowrie SSH/Telnet honeypot on an isolated AWS VPC entirely from Terraform, split into two stacks so the honeypot can be destroyed and rebuilt while the collected logs stay put
- ◇ Contained a host treated as already compromised by running Cowrie in a container with capabilities dropped and no metadata access, and denying all egress except S3
- ◇ Kept the real admin path out of the trap by moving sshd to a high port, key-only and reachable from a single admin IP
- ◇ Shipped every event off-box with Vector to a versioned, write-only S3 bucket as gzipped NDJSON in Hive-style partitions, ready to query from Athena or DuckDB, with budget alerts at 80% forecast and 100% actual spend
- ◇ Next: a FastAPI ingester into PostgreSQL with GeoIP, ASN, and reputation enrichment, a live MapLibre attack map, and HDBSCAN clustering to group attacker campaigns

Projects – AI/ML Engineering

P&ID Digitizer: Scanned Engineering Drawings to Graphs | *PyTorch, OpenCV, FastAPI, Docker* **2026**

- ◇ Converted raster piping and instrumentation diagrams into graphs (GraphML and DEXPI 2.0.1 XML), reaching 0.915 edge F1 and 0.997 symbol mAP@0.5 on a 75-sheet test split, by chaining a symbol detector, orthogonal line extraction, and graph assembly
- ◇ Detected small symbols on 7,000 px sheets by tiling into 1024^2 crops with 256 px overlap and merging boxes class-aware across tile seams, fine-tuning a torchvision Faster R-CNN in under 2 GPU-hours on a laptop RTX 4070
- ◇ Made predictions reviewable by calibrating per-edge confidence scores to observed accuracy and building a review UI that filters to low-confidence edges for relabeling, deleting, confirming, or adding
- ◇ Shipped it as a service: a FastAPI job API with SQLite job state, upload size and type limits, timeouts, and a SHA-1-checked model, packaged as a non-root CPU Docker image that runs end to end with `make demo`
- ◇ Kept the evaluation honest by running each test split exactly once and enforcing ruff, strict mypy, and 99 pytest cases in CI, with 26 design decisions recorded as they were made
- ◇ Found through error analysis that 80% of the remaining misses were nodes 2–4 px off rather than topology errors, and reported the real-drawing gap openly (0.49 edge F1 on 8 real sheets) along with the next fix to try

Word2Eq: Reproducible Seq2Seq Training & Inference Platform | *PyTorch, FastAPI* **2024, rebuilt 2026**

- ◇ Rebuilt a non-functional 2024 prototype into an end-to-end train, evaluate, and serve platform for a 6.4M-parameter seq2seq Transformer that trains in under 9 minutes on CPU
- ◇ Eliminated malformed outputs (97.4% → 100% valid equations on 1,000 SVAMP problems) by masking logits with a pushdown-automaton grammar during decoding, then checking every answer symbolically before returning it
- ◇ Made training preemption-safe, with resumed runs producing bit-identical weights, by checkpointing optimizer, scheduler, RNG, and in-epoch position atomically on SIGTERM, verified by 32 pytest cases in CI
- ◇ Made every model traceable to its inputs with versioned artifacts whose model cards record a config fingerprint, SHA-256 data hashes, and the git revision
- ◇ Served 150 requests/s at 73 ms p95 on one laptop CPU by micro-batching requests on a dedicated inference thread behind a bounded queue that sheds overload with a fast 503 and Retry-After
- ◇ Guarded releases with a CI promotion gate on accuracy and validity, and ran the service against availability, latency, and answer-integrity SLOs with burn-rate alerts, a runbook, and a Kubernetes manifest with probes, a PDB, and an HPA

The Limits of Foresight: Depth-Limited Reasoning in Go and Poker | *Python, LaTeX* **2026 – In Progress**

- ◇ Framed a question two research literatures rarely test together: whether people stop thinking ahead the same way about board positions (Go) and about other minds (poker)
- ◇ Designed two small, interpretable models that share one architecture and one depth parameter, so depth can be compared directly across domains
- ◇ Set four testable hypotheses: depth grows with player rank, belief depth tops out at about 2 levels, people spend more depth when stakes are higher, and returns diminish faster for minds than for boards
- ◇ Planned to fit both models to public human game data, using existing engines only as move evaluators and data sources, not as opponents
- ◇ Grounded the design in bounded rationality, level-k and cognitive-hierarchy models, and resource-rational analysis
- ◇ Status: pitch milestone for CS 6795 (Cognitive Science) at Georgia Tech, fall 2026; model fitting and results still to come

Projects – Tooling, Data & Other

DEXPI P&ID Viewer | *JavaScript, Python, XML, SVG* **2026**

- ◇ Built a single-page viewer and validator for DEXPI 2.0.1 P&ID files with zero dependencies, so anyone can open a file from disk and inspect it with nothing uploaded
- ◇ Drew diagrams that have no stored coordinates by laying them out from topology alone, with a Sugiyama-style layered flow layout, a stress-majorization network layout, and an auto mode that switches per connected group
- ◇ Caught structural errors before they reach downstream tools by checking well-formedness, ID syntax and uniqueness, and IDREF resolution
- ◇ Added metamodel checks against the Core and Plant 2.0.1 specs (class existence and abstractness, property kinds, target types, multiplicity bounds) by generating a compact class table from the spec XML with a Python build script
- ◇ Gave pipelines a headless path with a Python CLI validator (lxml) that runs both XSD and metamodel checks
- ◇ Made large models navigable with equipment, valve, and instrument symbols, a reference inspector, a model tree, ID and tag search, SVG export, and light and dark themes

The $2 Billion Patch Note: Market Event Study | *Python, pandas, SciPy* **2026 – In Progress**

- ◇ Treated Counter-Strike 2's October 2025 trade-up update, which erased an estimated $2 billion of item value in 48 hours, as a real market shock to be measured
- ◇ Measured the crash shape across about 3,900 items (a 19% drawdown, trough at 25 hours, half the loss recovered 48 hours later) by building price indices from daily Steam market medians
- ◇ Showed that newly useful Covert inputs gained 43.5% mean abnormal return within 48 hours ($t = 8.95$) while Restricted and Classified items fell, using a constant-mean event study per rarity group
- ◇ Found that the cheapest exteriors repriced most (Battle-Scarred Coverts up 147% at 2 days and 258% at 14 days, versus Factory New down 24%) by splitting abnormal returns by wear and StatTrak
- ◇ Modeled the new contract as a call option (five Coverts as the strike, a knife from the collection's pool as the payoff) with expected-value, Monte Carlo, and stress-test views
- ◇ Packaged the analysis as a tested Python CLI that regenerates every table and figure

The Thousand and One Words: SAT Vocabulary Comic Epic | *Python, HTML, SVG* **2026**

- ◇ Wrote a 100-chapter comic epic for readers aged 14 to 16 that teaches the 1,000 most-used SAT words, ten words per chapter
- ◇ Gave every word three memory hooks (a story scene, a picture link, and a memory-palace station), and chained all 1,000 words into one unbroken link chain across chapters
- ◇ Mapped the words onto a memory palace of 100 landmarks in the Round City of Baghdad, each with the same 10 stations
- ◇ Generated all 100 chapter pages, image-prompt files, and the combined script from one source with a Python build, so a one-line edit updates every page
- ◇ Drew the character sheets and city map as vector SVG in Python

Technical Skills

Languages: Python, JavaScript, Go, C/C++, SQL, NoSQL, Bash, HTML, CSS, LaTeX
Frameworks & Libraries: PyTorch, torchvision, OpenCV, LangChain, LlamaParse, FastAPI, NetworkX, pandas, NumPy, SciPy, Matplotlib, pytest, Jekyll
Tools & Platforms: Linux (Debian), Git, Docker, Kubernetes, Jenkins, Terraform, Vagrant, AWS, GitHub Actions, CI/CD, Prometheus, PostgreSQL, SQLite, Pinecone, OpenAI API, Autodesk Fusion
Security Tools: pfSense, Snort, Nessus, OpenVAS, Wireshark, PRTG, WireGuard, nftables, fail2ban, Cowrie, Vector

Organizations

Mensa **2013 – Present**